Formulários

📂 Caminho: src/main/resources/templates/

Facilita na hora do Desenvolvimento

```
SpringBootProjeto03FormsApplication.java   application.properties   Pessoa.java   UserController.java
1  spring.application.name=SpringBoot_Projeto03_Forms
2
3  # Desabilita o cache do Thymeleaf para que as mudanças no HTML apareçam na hora
4  spring.thymeleaf.cache=false
```

📂 Caminho: src/main/java/com/example/SpringBoot_Projeto03_Forms/model/Pessoa.java

```
SpringBootProjeto03FormsApplication.java   application.properties   Pessoa.java   UserController.java
1  package com.example.SpringBoot_Projeto03_Forms.model;
2
3  public class Pessoa  3 usages  Maiquel
4  {
5      private String nome;  3 usages
6      private int idade;  3 usages
7
8      // Construtor (Para criar a pessoa com dados)
9      public Pessoa(String nome, int idade)  no usages  Maiquel
10     {
11         this.nome = nome;
12         this.idade = idade;
13     }
14
15     // Métodos Getter e Setter para usar nos Formularios
16     public String getNome() { return nome; }
20
21     public void setNome(String nome) { this.nome = nome; }
25
26     public int getIdade() { return idade; }
30
31     public void setIdade(int idade) { this.idade = idade; }
35 }
36
```

A classe **Pessoa.java** atua como o **Model** dentro da arquitetura MVC. Sua finalidade é estritamente técnica e estrutural:

1. Definição do Objeto (Model)

Sua função é representar uma entidade do mundo real (uma Pessoa) em formato de código. Ela serve como o **contêiner** que transportará os dados entre o navegador e o servidor.

2. Finalidade para o Formulário

Para o formulário, esta classe serve como um **espelho de dados**:

- **Mapeamento**: Cada campo de `input` no HTML (nome e idade) precisa de um atributo correspondente nesta classe.
- **Encapsulamento**: Garante que os dados trafeguem de forma organizada e protegida dentro de um único objeto, em vez de variáveis soltas.

3. Requisitos para Funcionamento

Para que esta classe cumpra sua finalidade no formulário, ela deve possuir:

- **Getters**: Necessários para o Thymeleaf ler os dados e exibi-los na tabela.
- **Setters**: Necessários para o Spring Boot gravar os dados vindos do formulário dentro do objeto.
- **Construtor Padrão**: Necessário para que o framework consiga instanciar a classe automaticamente ao receber um `POST`.

📂 Caminho:
`src/main/java/com/example/SpringBoot_Projeto03_Forms/controller/UserController.java`

SpringBootProjeto03FormsApplication.java | application.properties | Pessoa.java | UserController.java

```java
package com.example.SpringBoot_Projeto03_Forms.controller;

import ...

@Controller  Maiquel
public class UserController
{

    // Lista do tipo Pessoa que simula um banco de dados em memória
    private List<Pessoa> listaPessoas = new ArrayList<>();  2 usages

    // 1. Rota GET: Responsável por carregar/exibir a página no navegador
    @GetMapping("/usuarios")  Maiquel
    public String exibirPagina(Model model)
    {
        // Envia a lista de pessoas para o HTML (Thymeleaf)
        // "listaPessoasHTML" é o nome que será usado no arquivo HTML
        model.addAttribute( attributeName: "listaPessoasHTML", listaPessoas);

        // // Retorna o arquivo HTML: templates/user/index.html
        return "user/index";
    }

    // 2. Rota POST: Recebe os dados enviados pelo formulário
    @PostMapping("/cadastrar")  Maiquel
    public String cadastrarPessoa(Pessoa pessoa)        // É o Pessoa input do Java
    {
        // Adiciona a pessoa na lista (simulando salvar no banco)
        listaPessoas.add(pessoa);

        // Redireciona para a rota GET acima para atualizar a visualização da lista
        return "redirect:/usuarios";
    }
}

// Toda vez que tá chamando a rota "/usuarios" está chamando a função exibirPagina(Model model) no arquivo
// UserController, que é função que renderiza a página HTML
```

A arquitetura do **UserController.java** baseia-se na definição de métodos especializados, tecnicamente denominados **Rotas (Routes)** ou **Endpoints**. Cada rota é vinculada a uma URL específica e a um verbo HTTP (GET ou POST).

1. Definição de Rotas (Mapeamento)

No Spring Boot, as funções não são executadas aleatoriamente; elas são "penduradas" em endereços web através de anotações. **Rota** é o caminho que o navegador percorre para acionar uma lógica no servidor.

- **@GetMapping("/usuarios")**: Define a **Rota de Visualização**. É o endereço que o usuário digita no navegador para acessar o sistema.
- **@PostMapping("/cadastrar")**: Define a **Rota de Ação**. É o endereço invisível para onde o formulário envia os dados ao clicar no botão "Cadastrar".

---

2. Função de Renderização: exibirPagina

Esta função tem como objetivo único construir a interface que o usuário verá.

- **Finalidade**: Localizar o arquivo físico index.html dentro da pasta templates/user e entregá-lo ao navegador.
- **Envio da Lista**: Para que o HTML não apareça vazio, esta função utiliza o objeto Model para "injetar" a **listaPessoas** (armazenada na memória do servidor) para dentro do Front-end.
- **Identificação**: A lista é enviada com o rótulo **"listaPessoasHTML"**. Este é o nome que o Thymeleaf utilizará no HTML para identificar os dados e montar a tabela.

3. Função de Processamento: cadastrarPessoa

Esta função é ativada apenas quando há um envio de dados via formulário.

- **Finalidade**: Capturar as informações digitadas pelo usuário e processar a lógica de negócio (neste caso, salvar o novo registro).
- **Recebimento de Dados**: Ela recebe o objeto **Pessoa** já preenchido. O Spring Boot realiza o trabalho de converter os campos de texto do formulário em um objeto Java pronto para uso.
- **Atualização do Fluxo**: Após adicionar o novo usuário à lista, a função não carrega um novo HTML; ela emite um comando de **Redirecionamento** para a Rota GET (/usuarios), garantindo que a página seja atualizada com o novo dado inserido.

Resumo Técnico da Comunicação

A **Rota GET** prepara o terreno e entrega a lista de dados ao HTML. A **Rota POST** recebe os dados novos, atualiza a lista e devolve o controle para a Rota GET, fechando o ciclo de interação do usuário.

OBS: Todas as funções do controller são nomeada através de uma rota.

📂 Caminho:
src/main/resources/templates/user/index.html

tProjeto03FormsApplication.java | application.properties | Pessoa.java | UserController.java | index.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Gerenciamento de Usuários</title>
</head>

<body>
    <h2>Cadastrar Novo Usuário</h2>

    <!-- Formulário que envia os dados para o Controller -->
    <!-- th:action="@{/cadastrar}" chama a rota POST "/cadastrar" -->
    <!-- Essa rota é tratada no método cadastrarPessoa(Pessoa pessoa) no UserController -->
    <form th:action = "@{/cadastrar}" method="post">

        <label>Nome:</label>
        <!-- name="nome" precisa ser igual ao atributo da classe Pessoa -->
        <!-- O Spring automaticamente pega esse valor e preenche o objeto Pessoa -->
        <input type="text" name="nome" placeholder="Digite seu nome" required>

        <label>Idade:</label>
        <!-- Mesmo conceito: name="idade" será enviado para o objeto Pessoa -->
        <input type="number" name="idade" placeholder="Digite sua idade" required>

        <button type="submit">Adicionar Usuário</button>

    </form>
```

```html
    <hr>

    <table border="1">

        <thead>
            <tr>
                <th>Nome</th>
                <th>Idade</th>
            </tr>
        </thead>
        <tbody>
            <!-- th:each percorre a lista enviada pelo Controller -->
            <!-- "listaPessoasHTML" foi adicionada no model no método exibirPagina() -->
            <!-- Para cada item da lista, ele cria uma linha da tabela -->
            <tr th:each="p : ${listaPessoasHTML}">

                <!-- th:text substitui o conteúdo da tag pelo valor do atributo -->
                <!-- p.nome vem da classe Pessoa -->
                <td th:text="${p.nome}"></td>

                <!-- p.idade também vem da classe Pessoa -->
                <td th:text="${p.idade}"></td>
            </tr>
        </tbody>

    </table>
</body>
</html>
```

A comunicação entre o **Backend (Java)** e o **Frontend (HTML)** é mediada pelo motor de templates **Thymeleaf**. Esta integração ocorre através de atributos técnicos inseridos diretamente nas tags HTML, identificados pelo prefixo `th:`.

Diferente de outras tecnologias que inserem blocos de código no meio do HTML, o Thymeleaf utiliza a própria estrutura das tags para executar lógica Java.

Análise Técnica das Tags Thymeleaf

1. `th:action="@{/cadastrar}"`

- **Função**: Mapeamento de destino do formulário.
- **Ação**: Esta tag conecta o formulário à **Rota POST** definida no `UserController`. Quando o usuário aciona o botão "submit", o Thymeleaf garante que os dados sejam enviados exatamente para o método `cadastrarPessoa` no servidor.

2. **Atributos `name="nome"` e `name="idade"`**

   - **Função**: Identificadores para o **Data Binding**.
   - **Ação**: Embora sejam atributos padrão do HTML, o Spring Boot os utiliza como "chaves". O valor digitado no campo com `name="nome"` é injetado diretamente no atributo `nome` da classe `Pessoa` através do método **Setter** correspondente.

3. **`th:each="p : ${listaPessoasHTML}"`**

   - **Função**: Iteração de coleções (Loop).
   - **Ação**: Esta tag localiza a lista enviada pelo Controller através do identificador **`${listaPessoasHTML}`**. Para cada objeto presente na lista, o Thymeleaf duplica a tag `<tr>` (linha da tabela), criando uma representação visual para cada registro armazenado na memória do Java.

4. **`th:text="${p.nome}"` e `th:text="${p.idade}"`**

   - **Função**: Saída de dados e renderização de texto.
   - **Ação**: Esta tag limpa qualquer conteúdo estático que estiver dentro da `<td>` e o substitui pelo valor real vindo do objeto Java. Para realizar esta operação, o Thymeleaf chama internamente os métodos **Getters** (`getNome()` e `getIdade()`) da classe `Pessoa`.

Resumo da Comunicação por Tags

A integração funciona como um contrato de atribuição:

1. As tags de **envio** (`th:action` e `name`) levam a informação do formulário para os **Setters** no Java.
2. As tags de **exibição** (`th:each` e `th:text`) buscam as informações dos **Getters** no Java e as escrevem no HTML.

Diferenças do Spring Boot para o .Net

1 - Todas as funções do Controller são nomeadas e ativada por uma rota

Nomeando a função cadastrarPessoa(Pessoa pessoa) por um Rota chamada "/cadastrar"

```
31      // 2. Rota POST: Recebe os dados enviados pelo formulário
32      @PostMapping("/cadastrar")  Maiquel
33      public String cadastrarPessoa(Pessoa pessoa)        // É o Pessoa input do Java
34      {
35          // Adiciona a pessoa na lista (simulando salvar no banco)
36          listaPessoas.add(pessoa);
37
38          // Redireciona para a rota GET acima para atualizar a visualização da lista
39          return "redirect:/usuarios";
40      }
```

Formulário ligado a rota "/cadastrar" que ativa a função cadastrarPessoa(Pessoa pessoa) no controller

```
14          <form th:action = "@{/cadastrar}" method="post">
```

2 - A comunicação com Java do Front End é feita dentro das TAGS HTML

Lista no controller enviando para o arquivo HTML

```
23          // Envia a lista de pessoas para o HTML (Thymeleaf)
24          // "listaPessoasHTML" é o nome que será usado no arquivo HTML
25          model.addAttribute( attributeName: "listaPessoasHTML", listaPessoas);
```

Acessando essa lista dentro da Tag HTML

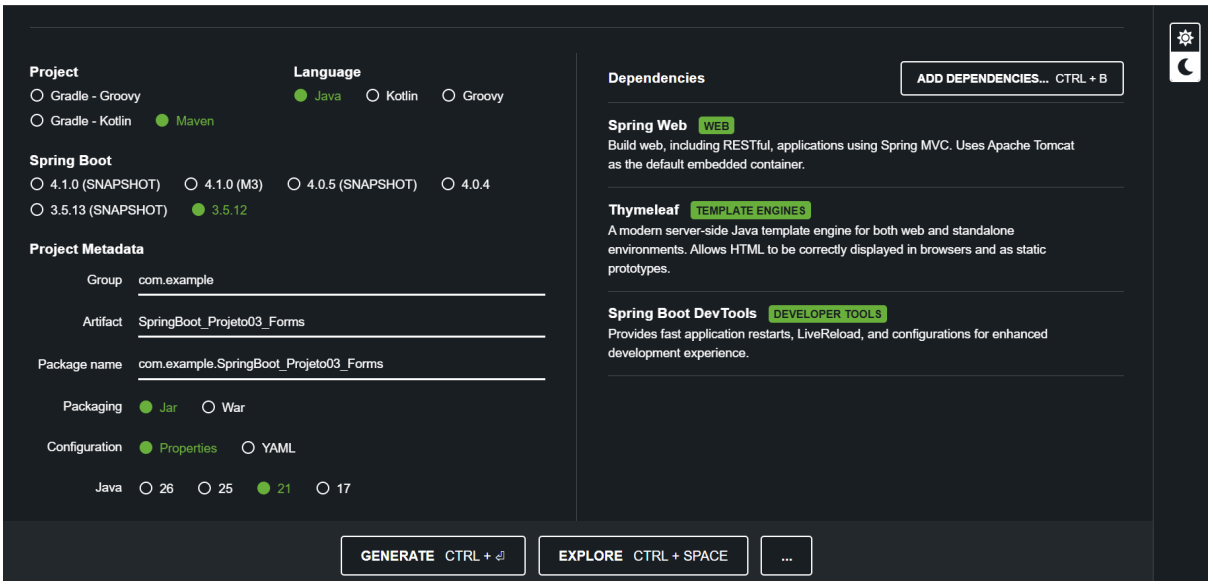
```
<!-- th:each percorre a lista enviada pelo Controller -->
<!-- "listaPessoasHTML" foi adicionada no model no método exibirPagina() -->
<!-- Para cada item da lista, ele cria uma linha da tabela -->
<tr th:each="p : ${listaPessoasHTML}">

    <!-- th:text substitui o conteúdo da tag pelo valor do atributo -->
    <!-- p.nome vem da classe Pessoa -->
    <td th:text="${p.nome}"></td>

    <!-- p.idade também vem da classe Pessoa -->
    <td th:text="${p.idade}"></td>
</tr>
```

As **rotas** do .Net usamos o nome das funções criada no controller e na comunicação do **front end** precisamos quebrar o bloco usando @{...} para escrever código C#

Configuração Inicial do Projeto



1. Spring Web

É a dependência central para o desenvolvimento de aplicações voltadas à web.

- **Finalidade**: Fornece o suporte completo para a criação de aplicações **Spring MVC** (Model-View-Controller) e serviços **RESTful**.
- **Componente Embutido**: Inclui automaticamente o **Apache Tomcat**, que funciona como o servidor de aplicações interno. Isso significa que você não precisa instalar um servidor externo para rodar o seu projeto; o Java inicia o servidor por conta própria.
- **Responsabilidade**: Gerenciar as requisições HTTP, as rotas (`@GetMapping`, `@PostMapping`) e a comunicação entre o navegador e o seu Controller.

---

2. Thymeleaf

É o motor de templates (**Template Engine**) utilizado para processar o Front-end no lado do servidor.

- **Finalidade**: Atuar como a ponte de comunicação entre o Java e o HTML.
- **Diferencial Técnico**: Permite a criação de **Natural Templates**. Isso possibilita que o arquivo HTML seja aberto e visualizado em um navegador comum como um protótipo estático, mas funcione dinamicamente quando processado pelo Spring Boot.
- **Responsabilidade**: Interpretar as tags com o prefixo `th:` (como `th:each` e `th:text`), injetando os dados vindos do Model diretamente na estrutura do HTML antes de enviá-lo ao usuário.

---

3. Spring Boot DevTools

É um conjunto de ferramentas utilitárias focadas na produtividade do desenvolvedor durante a fase de construção.

- **Finalidade**: Automatizar processos repetitivos de reinicialização e atualização.
- **Recursos Principais**:
  - **Automatic Restart**: Reinicia a aplicação automaticamente sempre que você altera e salva um arquivo no diretório `src/main/java`.
  - **LiveReload**: Atualiza o navegador automaticamente quando o Front-end (HTML/CSS) é modificado.
  - **Configurações de Desenvolvimento**: Desabilita automaticamente caches (como o do Thymeleaf) para garantir que as mudanças sejam vistas instantaneamente.
- **Responsabilidade**: Eliminar a necessidade de parar e subir o servidor manualmente a cada pequena alteração no código.